UNIT 1, FOUNDATIONS OF AI AGENTS AND MICROSOFT FOUNDRY

LECTURE 3

Conversational AI Fundamentals

What actually gets sent on every request, why a long chat quietly gets more expensive and less accurate at the same time, and what to do about it.

CSE476 | Rohit Bharti

SECTION 00

Where we left off

Two facts from the last two lectures, both load bearing today.

RECALL

Two things you already know

FROM LECTURE 1

The model remembers nothing

Not one word between calls. Every request is the first request as far as it is concerned. The illusion of memory is you, resending the transcript.

FROM LECTURE 2

The model is a reflex agent

One percept, one action. State is not a model feature. It is a list you maintain in your own code.

PUT THEM TOGETHER AND SOMETHING UNCOMFORTABLE FALLS OUT

If memory is a list you resend, then a conversation is not a stream. It is the same document, sent again and again, getting longer every time. Today is about what that actually costs you and what it does to the answers.

Today

- What a turn actually is, and the four roles in every transcript.
- Why cost grows faster than the conversation does, with the arithmetic in front of you.
- Context windows, what happens at the limit, and the accuracy problem that starts well before it.
- What belongs in a system prompt and what does not.
- Three ways to keep a transcript from exploding, and what each one loses.
- A live build where you measure a real conversation, break it, and fix it.

HOLD ON TO THIS

Practical 2 requires a multi turn agent. Everything in this lecture is the difference between a demo that works for three turns and something that survives thirty.

SECTION 01

What a conversation actually is

Four roles and one uncomfortable arithmetic.

Four roles. That is the whole vocabulary.

system	**The job description**	Who the agent is, what it may and may not do. Set once, sent every time.
user	**What the person said**	The request. In an agentic setting this is a goal, not a question.
assistant	**What the model said**	Either a final answer, or a request to use a tool. Both are just text.

THERE IS NO FIFTH ROLE

No hidden memory store, no session object, no server side thread you are attaching to. A conversation is a list of dictionaries with one of four values in the role key. That is the entire mechanism.

A turn is not one message

One thing the user asked. Five messages on the wire.

user	"Rooms and price at Taj Palace on 14 August?"
assistant	tool_calls: get_room_availability(...)
tool	"Taj Palace on 2026-08-14: 0 rooms available."
assistant	tool_calls: get_hotel_details(...)
tool	"Taj Palace: Rs 14500 per night, 2.1 km from campus."

SO WHEN SOMEBODY SAYS A TEN TURN CONVERSATION

They may mean ten messages, or they may mean sixty. In an agent with tools, one user request routinely produces five or six messages. Every one of them lives in the transcript forever, and every one of them gets resent.

The uncomfortable part

> **On turn ten, you do not send turn ten. You send turns one through ten.**

The model has no memory, so the transcript is the memory, so the transcript goes up the wire on every single request. Turn one sends one turn. Turn ten sends ten. Turn thirty sends thirty.

WHICH MEANS

Your conversation grows linearly. Your bill does not. Each turn is bigger than the last, so the total across a conversation grows with the square of its length. A thirty turn conversation does not cost three times a ten turn one. It costs closer to nine.

The arithmetic, with real numbers

turn	tokens sent on that request	total sent across the whole conversation
1	150	**150**
5	350	**1,250**
10	600	**3,750**
20	1,100	**12,500**
40	2,100	**44,000**

Fifty token turns, a modest system prompt, and no trimming. Four times the turns, twelve times the cost.

Measure it. Do not estimate it.

```
from cse476.conversation import transcript_tokens, tokenizer_is_exact

print(tokenizer_is_exact())        # True if a real tokenizer loaded
print(transcript_tokens(messages)) # what this request will cost you
```

THE HABIT

Print it every turn

One line in your loop. You will spot a runaway transcript in seconds rather than on an invoice at the end of the month.

THE CAVEAT

Tokenizers differ by model

A count that is exact for GPT is approximate for Llama. Check tokenizer_is_exact before you quote a number to anyone.

HOLD ON TO THIS

Anything you cannot measure, you will argue about. Anything you can measure, you can put in a graph and settle.

SECTION 02

The context window

A hard limit, and a soft problem that starts long before it.

THE CONTEXT WINDOW

The layman version

Think of a desk with a fixed surface area.

Every document you want the model to consider has to be lying on that desk at the same time. The system prompt is on it. Every message so far is on it. Every tool result is on it. The desk does not get bigger, and when it is full, something has to come off.

The model cannot glance at a filing cabinet. If it is not on the desk, it does not exist.

THE TECHNICAL CORRELATE

The context window is the maximum number of tokens the model can attend to in one request, counting your input and its output together. On the free GitHub Models lane it is roughly 8,000 in and 4,000 out, which is generous for learning and tight for anything real.

What actually happens when you exceed it

THE API REFUSES	Most providers return a 400 with a context length message. Loud, immediate, easy to fix.
SILENT TRUNCATION	Some stacks quietly drop the oldest messages for you. Your agent forgets things and nothing errors. This is the dangerous one.
DEGRADED, NOT BROKEN	You stay under the limit but accuracy has already fallen. No error at all. Covered on the next slide.

THE ONE TO FEAR IS THE MIDDLE ROW

An error you can see is a bug you will fix this afternoon. An agent that silently forgot the customer's allergy is a bug you find out about from the customer. Know which behaviour your stack has, and if you are not sure, test it deliberately.

Tokens are not words

ENGLISH PROSE

Roughly 4 characters

A rule of thumb that is close enough for planning and wrong often enough that you should still measure.

CODE AND JSON

Much worse

Punctuation, indentation and braces all cost tokens. A tool schema is expensive, and you send it on every single request.

INDIAN LANGUAGES

Worse again

Scripts outside the tokenizer's training distribution fragment into many more tokens per word. The same sentence in Hindi can cost several times what it costs in English.

A CONSEQUENCE WORTH SITTING WITH

The same product costs more to run for a Hindi speaking user than an English speaking one, purely because of how the tokenizer was built. That is a real fairness question, and it is the kind of thing Unit 6 asks you to notice rather than shrug at.

The problem that starts before the limit

Published research on long context models found something counterintuitive. Accuracy is not flat across the window.

BEGINNING	MIDDLE	END
best recall	noticeably worse	best recall

where the relevant fact sits in the transcript

WHY THIS MATTERS FOR AN AGENT

Your important context tends to end up exactly in the middle. The system prompt is at the start, the latest question is at the end, and the thing the user told you fifteen turns ago is buried in between. A bigger context window does not fix this. It gives you more middle.

Two curves, going opposite ways

AS THE CONVERSATION GROWS

Cost goes up, fast

Every turn resends everything before it. The total grows with the square of the length, not the length.

AT THE SAME TIME

Accuracy drifts down

More material to attend to, and the older facts are sitting in the least reliable part of the window.

So a longer conversation is not just more expensive. It is more expensive and worse.

Which is why the rest of this lecture is about deliberately throwing things away.

SECTION 03

The system prompt

Sent on every request, so every word in it is a recurring cost.

What belongs in a system prompt

- Who the agent is and what job it does. One or two sentences, not a biography.
- What it must not do. Refusals, scope limits, anything it should decline.
- How to use the tools it has, when that is not obvious from the schemas.
- The format of the final answer, if you need a specific one.
- What to do when it cannot answer. This one is skipped constantly and it is the difference between an honest failure and a confident invention.

THE CONSTRAINT NOBODY MENTIONS

This goes up the wire on every single request, forever. A three hundred word system prompt on a forty turn conversation is twelve thousand words of pure repetition. Every sentence should be earning that, and most long system prompts contain several that are not.

What does not belong

NOT HERE

Data that changes

Today's prices, current availability, this user's booking history. That is what tools are for. A system prompt is a job description, not a database.

NOT HERE

Anything secret

Keys, internal URLs, business rules you would not show a customer. Users extract system prompts routinely, and Unit 6 shows you how easily.

NOT HERE

Long examples

Every example is paid for on every turn. If you need many, that is a retrieval problem, not a prompting one.

THE TEST

Would you be comfortable if this exact text appeared in your product's public documentation? If not, it does not belong in a system prompt, because that is roughly the level of protection it has.

Instruction drift

The system prompt sits at position zero. Twenty turns later it is a long way from where the model is paying most attention.

THE SYMPTOM

It stops obeying rules it obeyed at turn three

The tone slips. A constraint you set gets ignored. It starts answering things you told it to decline. Nothing errored, and your prompt did not change.

THE CAUSE

Distance and dilution

The instruction is still there, but it is now competing with thousands of tokens of conversation, sitting in the part of the window with the weakest recall.

THE PRACTICAL FIX

Reassert critical constraints near the end of the transcript rather than only at the start, or keep the conversation short enough that drift never begins. The second is usually cheaper and almost nobody tries it first.

The misconception

WHAT PEOPLE DO

Just add it to the system prompt

Every new edge case becomes another sentence. After a month it is nine hundred words, nobody knows which lines still matter, and it is billed on every turn.

WHAT USUALLY WORKS BETTER

Move it into code

A rule you can enforce in Python is enforced every time. A rule in a prompt is a request the model usually honours. When correctness matters, the difference is not subtle.

HOLD ON TO THIS

You already saw this in Lecture 1. The reflex agent was held to one tool call by max_steps, not by the sentence asking it politely. A prompt is a request. Code is a guarantee.

SECTION 04

Keeping it under control

Three strategies, and what each one throws away.

Three strategies. All three lose something.

SLIDING WINDOW — Keep the system message and the most recent messages that fit.

SUMMARISATION — Replace older turns with a short summary of them.

PINNING — Mark certain messages as never droppable.

THERE IS NO FOURTH OPTION CALLED KEEP EVERYTHING

That is the strategy you have been using so far, and it is the one that produces the cost curve from section one. Doing nothing is a choice, and it is usually the worst one.

Sliding window, and the trap in it

```
trimmed = sliding_window(messages, max_tokens=2000)

# keeps: the system message, then as many recent
#        messages as fit inside the budget
```

THE TRAP, AND IT WILL BITE YOU

A naive slice can cut between an assistant tool call and the tool result that answers it. That leaves an orphaned tool message, and most providers reject the whole request with an error that does not mention orphans at all. Our implementation drops orphans on the way out, and there is a test that proves it.

HOLD ON TO THIS

Use it when the useful context is recent, which for a booking assistant it usually is. Do not use it when the user told you something important twenty turns ago.

Summarisation

```
compact = summarise_older(
    messages,
    keep_recent=6,
    summariser=lambda text: ask_model_to_summarise(text),
)
```

WHAT IT BUYS

The gist, cheaply

Thirty turns of history compressed into a paragraph. The agent still knows roughly what happened.

WHAT IT COSTS

Precision, and a call

Exact wording is gone. Never summarise a booking reference or a price the user agreed to.

HOLD ON TO THIS

A summary is a lossy compression of your user's conversation. Compress the chat, not the commitments.

Pinning, and why it is not optional

Some facts must survive any amount of trimming. Sliding windows do not know which.

```
messages.append(pin({"role": "user",
                     "content": "I am allergic to peanuts"}))

kept = sliding_window_with_pins(messages, max_tokens=2000)
```

THIS IS TESTED, AND THE TEST IS THE ARGUMENT

tests/mock_run_l3.py runs the same thirty turn conversation through both functions. The pinned version keeps the allergy. The plain sliding window drops it, silently, with no error. Same budget, same data, one line of difference.

HOLD ON TO THIS

Allergies, budgets, booking references, accessibility needs, anything stated once and relied on later. If losing it would be a serious failure, it does not belong in a droppable message.

SECTION 05

Live build

Measure it, break it, fix it, then see what the fix cost you.

What we are doing

1	**Measure**	Print the transcript size on every turn of a real conversation. Watch it climb.
2	**Model the cost**	Run the offline simulation. Forty turns, no API calls, and the shape of the curve.
3	**Trim it**	Apply a sliding window. Watch the cost fall.
4	**Find what you broke**	Ask the agent something only an old turn could answer. Watch it fail.

OPEN THIS NOW

notebooks/u1/l3_conversation.ipynb Steps 2 and 5 run entirely offline, so they cost nothing even if your lane is rate limited.

Step 1, make the invisible visible

```
for turn in conversation:
    messages.append(turn)
    print(f"turn {n}: sending {transcript_tokens(messages)} tokens")
```

One line. It is the cheapest debugging tool in this entire course, and it takes about four seconds to add.

WHAT YOU WILL SEE

The number never goes down. It cannot, because nothing removes anything. That monotonic climb is the whole problem stated in one column of output, and it is far more convincing than any slide of mine.

Step 4, the part that matters

After trimming, ask the agent something only an early turn could answer.

> *"What was the budget I mentioned at the start?"*

NOTE HOW IT FAILS

It does not say "I no longer have that". It answers, confidently, with something plausible and wrong, because a model asked a question with no supporting context will still produce fluent text. That is the behaviour to be afraid of, and it is why Unit 5 spends a whole block on hallucination detection.

HOLD ON TO THIS

Trimming did not cause a crash. It caused a lie. Those are very different bugs and only one of them shows up in your logs.

Hold on to this

A conversation is one document, resent every turn, getting longer and less reliable as it goes.

Cost grows with the square	Forty turns costs roughly twelve times what ten turns costs.
Accuracy falls before the limit	The middle of a long transcript is the least reliable part of it.
A prompt is a request, code is a guarantee	If correctness matters, enforce it in Python.
Trimming causes lies, not crashes	And a lie does not appear in your logs.

Before next session

DO	**Practical 2, due**	The multi turn hotel agent. Your trimming strategy and your justification for it are now part of the marks.
DO	**Measure your own agent**	Add the token print to your Practical 2 loop. Include a screenshot of the growth in your submission.
THINK	**Pick your pins**	For your agent, list three facts that must never be dropped. Be ready to defend why those three.

ON THE THIRD ONE

Most people list things that are merely useful. The question is narrower than that. Which facts, if silently lost, would make your agent give a confidently wrong answer rather than an unhelpful one? Those are the pins.

Next session

Unit 1, Lecture 4

Planning and Reasoning

The longest lecture in the unit, and the one you will use for the rest of your career. ReAct properly, why reasoning before acting beats acting first, how a plan survives contact with a failed tool call, and the failure modes that make an agent loop forever while looking productive.

COME WITH

Practical 2 submitted, your token growth screenshot, and your three pins.